



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 4

Team No.: 17

Project Title: Linux Process Creation, Execution and Termination System

Team Members

Roll Number	Student Name	Signature
2520030605	T. B. S SUNIL	
2520030479	V. PREM KUMAR	
2520030481	G. B. VEDADITYA	

1. Abstract

The Linux Process Creation, Execution and Termination System is a Linux-based project designed to demonstrate the complete life cycle of a process. The project focuses on how Linux creates a child process, executes a program, synchronizes parent and child processes, and terminates the process correctly. It applies Operating Systems and Systems Programming concepts through standard Linux system calls such as `fork()`, `exec()`, `wait()`, `waitpid()`, and `exit()`. The system can display process information including Process ID (PID), Parent Process ID (PPID), and process status, making the process relationship easier to observe. It also provides a practical way to understand process synchronization, exit status, and resource cleanup, and can be used to illustrate zombie and orphan process concepts.

The proposed solution will be implemented in C on Linux and will provide a simple, structured demonstration of process creation, execution, synchronization, and termination. The expected outcome is a working application that connects Operating Systems theory with actual Linux process-management mechanisms and makes the process life cycle easy to understand and explain.

2. Problem Statement

Process management is a fundamental Operating Systems concept, but the relationship between process creation, program execution, synchronization, and termination can be difficult to understand through theory alone. The project addresses this problem by providing a simple Linux-based practical system that demonstrates the process life cycle using real Linux system calls. It shows how a parent process creates a child, how the child executes a program, how the parent waits for the child, and how termination status is handled. This makes the concepts directly observable and relevant to Systems Programming.

3. Objectives

1. To understand and demonstrate the complete Linux process life cycle.
2. To create child processes using `fork()` and execute programs using the `exec()` family.
3. To synchronize and monitor parent-child processes using `wait()` or `waitpid()`.
4. To demonstrate process termination, exit status, PID/PPID information, and proper cleanup.

4. Proposed Methodology

The system will be implemented in C on Linux. First, the user provides a command or program as input. The parent process calls `fork()` to create a child process. The child uses an appropriate `exec()` system call to execute the requested program, while the parent uses `wait()` or `waitpid()` to synchronize with the child and collect its termination status. PID and PPID information will be displayed to show the parent-child relationship. Finally, `exit()` is used for controlled termination and the result is reported.

Flow: Input → `fork()` → Child/Parent → `exec()` → Execution → `wait()/waitpid()` → exit/status → Completion.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation — fork()	Creates a new child process from the parent process.
Program Execution — exec() family	Replaces the child process image with the requested program.
Process Synchronization — wait()	Allows the parent to wait for a child process to terminate.
Process Monitoring — waitpid()	Allows the parent to wait for and monitor a specific child process.
Process Termination — exit()	Terminates a process and provides an exit status.
Process Identification — getpid(), getppid()	Obtains the current process ID and parent process ID.
Process Lifecycle	Demonstrates parent-child relationships, state transitions, synchronization, and termination.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030605	T. B. S SUNIL	Process creation using fork() and program execution using the exec() family.
2520030479	V. PREM KUMAR	Parent-child synchronization using wait()/waitpid() and process termination handling.
2520030481	G. B. VEDADITYA	PID/PPID display, process-status observation, testing, debugging and documentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and development environment.
C	Programming language used to implement the project.
GCC	Compiler used to build the C program.
Linux system calls / POSIX APIs	Process creation, execution, synchronization, identification and termination.
Git / GitHub	Version control, source-code management and project repository.

8. Expected Outcome

The completed project is expected to provide a working Linux application that demonstrates process creation, program execution, parent-child synchronization, and process termination. It will display relevant process information such as PID and PPID and show the outcome of child-process execution. The project will provide a clear practical demonstration of Linux process management and help students connect Operating Systems concepts with real system calls.

9. GitHub Repository

GitHub Repository Link: [UnKnown-0703/KLH_CSE_2026-2027_TEAM-17_OSSP_Linux_Process_Creation_Execution_and_Termination_System-](https://github.com/UnKnown-0703/KLH_CSE_2026-2027_TEAM-17_OSSP_Linux_Process_Creation_Execution_and_Termination_System-17)

Repository Created: ☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Soumya Erukonda

Signature:

Remarks:

Faculty Use

Project Approved: ☐ Yes ☐ No

Date:

Faculty Signature: